Resource-Aware Scheduling Algorithms For Machine Learning in Heterogeneous Environments

Atharva Rajmane
Department of Information Technology
DJ Sanghvi College of Engineering
Mumbai, India
atharvarajmane53@gmail.com

Vedant Kesharia
Department of Information Technology
DJ Sanghvi College of Engineering
Mumbai, India
keshariavedant@gmail.com

Smit Shah
Department of Information Technology
DJ Sanghvi College of Engineering
Mumbai, India
smitshahofficial@gmail.com

Vinaya Sawant
Department of Information Technology
DJ Sanghvi College of Engineering
Mumbai, India
vinaya.sawant@djsce.ac.in

***Abstract*—In machine learning, efficiently managing diverse computational resources in heterogeneous environments—where nodes vary significantly in resources like CPUs, GPUs, and memory—poses unique challenges. Traditional scheduling algorithms, optimized for homogeneous systems, often lead to resource inefficiencies and increased latency in such setups. This paper introduces a resource-aware scheduling algorithm that leverages a master-slave architecture, where code snippets are divided into multiple sub-sections and distributed across nodes. Using multithreading, the master node dynamically allocates GPU resources based on each node's specific capabilities to optimize task execution. Experimental results demonstrate that our approach outperforms conventional methods, achieving higher resource utilization and lower task completion time, highlighting the potential of resource-aware scheduling to enhance the efficiency and scalability of distributed machine learning in complex, heterogeneous environments.**

***Index Terms*—CPU, distributed systems, GPU, heterogeneous environment, multithreading, parallel computing, scheduling algorithms**

I. Introduction

In recent years, machine learning (ML) has become essential for handling complex, large-scale data processing tasks. By distributing workloads across multiple computing nodes, ML systems can harness greater computational power, accelerate processing, and enhance scalability. However, with the growing use of heterogeneous environments—where nodes within a network possess varying resources such as CPUs, GPUs, memory capacities, and storage—new challenges arise. Unlike homogeneous environments, where resources across nodes are uniform, heterogeneous environments require specialized scheduling algorithms to efficiently allocate tasks according to each node's unique capabilities. This project aims to design and implement a resource-aware scheduling algorithm that optimizes resource allocation within these heterogeneous environments, reducing task completion time and improving overall system efficiency.

In heterogeneous distributed ML environments, resources vary widely across nodes. Some nodes may be optimized for computationally intensive tasks with high-performance GPUs, while others might offer substantial memory resources but limited processing power. Traditional scheduling algorithms, designed for uniform environments, are ineffective in these settings. They typically distribute workloads without regard for the specific resources of each node, often leading to resource underutilization, higher latency, and inefficiencies that detract from system performance. Consequently, there is a need for scheduling algorithms that can intelligently allocate tasks based on each node's distinct resource profile.

This project employs a master-slave architecture as the structural framework for task scheduling and distribution. In this setup, a central master node manages task distribution by breaking code snippets into smaller, independently executable sub-sections, which are then distributed to slave nodes. Each of these sub-sections is processed in parallel, allowing for efficient task execution across nodes. To further streamline resource allocation, the master node uses multithreading to simultaneously assess each slave node's computational capabilities—such as CPU speed, available memory, and GPU power—and dynamically allocates GPU resources accordingly. This approach enables adaptive scheduling, where tasks are matched to nodes based on real-time system load and specific resource requirements.

To evaluate the effectiveness of our proposed scheduling algorithm, we conducted experiments across a diverse range of machine learning workloads. In the homogeneous environment, we utilized a Convolutional Neural Network (CNN) trained on the CIFAR-10 dataset and a Reinforcement Learning (RL) model. These models were chosen for their computational intensity and their ability to simulate realistic training scenarios on uniform hardware (NVIDIA T4 GPU). In the heterogeneous environment, we tested our algorithm

using four representative Python scripts—simple, branching, looping, and parallelizable tasks—each designed to capture different execution patterns and computational dependencies. These scripts were executed across nodes with varied hardware capabilities, including systems with NVIDIA L4 GPUs, to evaluate the algorithm's adaptability and effectiveness in real-world distributed settings. These varied test cases provide a comprehensive basis for analyzing the impact of our resource-aware scheduling algorithm on execution time and resource utilization across both homogeneous and heterogeneous environments.

The primary goals of this project are to achieve efficient resource utilization by dynamically allocating tasks based on real-time conditions, thereby reducing latency and optimizing throughput. Key considerations include each node's computational power, current workload, and available resources within the network. For instance, GPU-intensive tasks such as convolutional operations within the CNN model are directed to nodes with powerful GPUs, while memory-intensive tasks or lightweight control logic are allocated to more appropriate nodes. Such intelligent scheduling ensures that each node operates at its optimal capacity, significantly enhancing the overall performance and efficiency of the distributed ML system.

This paper presents the following key contributions:

1) Development of a resource-aware scheduling algorithm that efficiently manages task distribution by leveraging a master-slave architecture combined with multithreading.
2) Implementation of a dynamic allocation strategy that continuously adapts to the heterogeneous system's resource variations, ensuring optimal task-to-node matching.
3) Comprehensive evaluation across diverse machine learning workloads and heterogeneous computing environments, demonstrating enhanced execution efficiency and improved utilization of computational resources.

II. Literature Survey

In designing resource-aware scheduling algorithms for heterogeneous environments, the work by Mao et al. (2019) on "Learning scheduling algorithms for data processing clusters" [1] provides a foundational approach that is highly relevant to this project. Mao and colleagues explore the use of reinforcement learning to optimize scheduling decisions within data processing clusters, focusing on dynamically adapting to changing workloads and resource conditions. Their work demonstrates that data-driven approaches can significantly improve task allocation efficiency compared to traditional rule-based methods. This insight is particularly useful for our project, as heterogeneous environments require adaptive scheduling strategies that account for varying CPU, GPU, and memory resources across nodes. By drawing from Mao et al.'s techniques for learning-based scheduling, our approach incorporates resource-specific decision-making processes, allowing for real-time allocation adjustments that improve task execution times and resource utilization across diverse computing environments.

About the Heterogeneous Environment , the research by Singh, Bhunia, and Veeravalli [7]on Heterogeneous Computing Systems provides crucial insights into optimizing task execution in environments with diverse computational resources. Their study explores strategies for balancing workloads across different processing units, ensuring efficient utilization of both high-performance and general-purpose computing elements. This is particularly relevant to our project, where resource-aware scheduling is essential to maximize efficiency in distributed machine learning tasks. By leveraging their methodologies, our approach aims to minimize execution bottlenecks and improve overall system performance. Their work serves as a valuable reference in designing adaptive scheduling techniques that dynamically allocate resources based on computational demand.

Another study on Heterogeneous computing systems by Topcuoglu, Hariri, and Wu (2014) on "A Resource-Aware Scheduling Algorithm with Reduced Task Duplication on Heterogeneous Computing Systems" [6] provides a strong foundation for optimizing resource utilization in our research. Their approach focuses on minimizing task duplication, which significantly improves execution efficiency and reduces unnecessary computational overhead in heterogeneous environments. This insight is particularly relevant to our project, as efficient scheduling is crucial when dealing with diverse computing resources like CPUs and GPUs. By incorporating their resource-aware scheduling techniques, our work aims to enhance task allocation strategies, ensuring better workload distribution and system performance. Their findings reinforce the need for adaptive scheduling methods that dynamically adjust to varying resource capacities, making it a valuable reference for our research.

The study by Das, Tiwari, and Agrawal (2020), titled "Dynamic resource allocation in cloud computing using a distributed scheduler," [2]offers valuable insights into designing distributed scheduling frameworks for dynamic resource environments, a concept highly applicable to this project. Their work investigates methods for dynamically allocating cloud resources by leveraging a distributed scheduling model, which efficiently manages varying workloads across a network of nodes. This paper is instrumental in shaping our approach to resource-aware scheduling in heterogeneous environments, as it highlights the importance of decentralized decision-making and real-time resource adjustments to optimize performance. By integrating principles from Das et al.'s distributed scheduling techniques, our project aims to implement an adaptive resource allocation system that can account for node-specific resource disparities, such as variations in CPU and GPU capacities, thereby enhancing scheduling flexibility and task completion efficiency in distributed machine learning setups.

The research by Vasilakos and Liu (2016) on "Scheduling

and resource allocation for real-time streaming services in cloud computing" [3] provides a critical perspective on achieving low-latency resource management, a concept that is particularly relevant to our project's focus on task scheduling in heterogeneous environments. Vasilakos and Liu address the need for efficient resource allocation in cloud-based real-time streaming services, proposing scheduling algorithms that minimize latency and maximize resource utilization under time-sensitive conditions. This paper's emphasis on balancing resource demand with real-time performance aligns with the goals of our research, as distributed machine learning tasks in heterogeneous environments often require swift allocation of resources to maintain system efficiency. Leveraging the low-latency scheduling strategies outlined by Vasilakos and Liu, our approach integrates similar techniques to enhance responsiveness, dynamically matching tasks to the most suitable nodes based on their resource profiles and minimizing delays in data processing workflows.

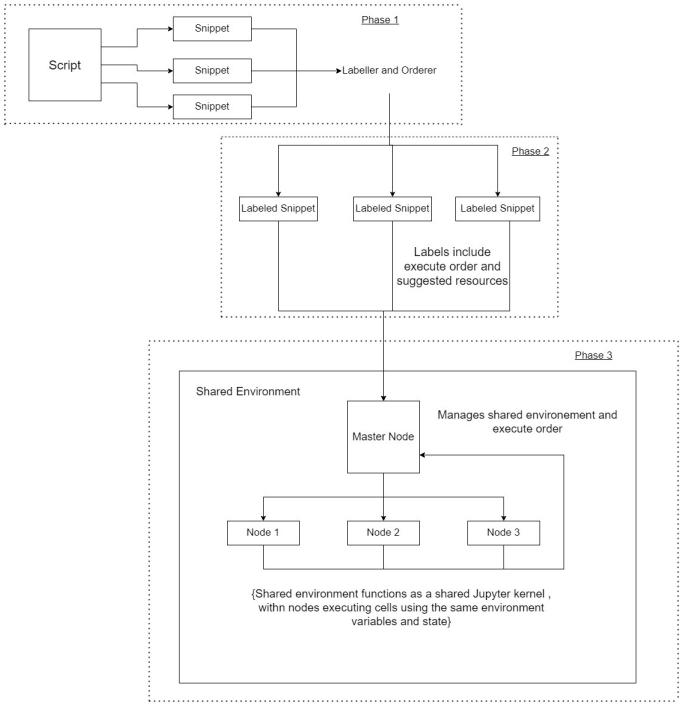
Additionally, Zeng et al. (2020), in their work "A container-based scheduling approach for heterogeneous cloud environments" [8], introduce a containerized resource allocation method that allows efficient scheduling for both high-performance and general-purpose tasks. Their research demonstrates how containerization can enable better resource sharing and scheduling flexibility, concepts that are directly applicable to heterogeneous distributed systems like ours. By isolating workloads and optimizing resource allocation at a granular level, their approach ensures that high-priority tasks are executed with minimal interference. This method also reduces overhead, making it particularly suitable for environments with diverse resource demands. Furthermore, their work highlights how container orchestration tools like Kubernetes can dynamically adjust resources in response to workload variations, ensuring seamless scalability. These insights are instrumental in refining our scheduling model to maintain performance consistency while handling real-time computational loads efficiently. By leveraging containerization, we can also enhance fault tolerance, ensuring that tasks continue executing smoothly even when certain nodes experience failures. This makes the scheduling process not just efficient but also more resilient and adaptable to real-world cloud computing challenges.

The review by Zhou, Tian, and Buyya (2021) titled "Deep Reinforcement Learning-based Methods for Resource Scheduling in Cloud Computing: A Review and Future Directions" [4] offers a comprehensive analysis of using deep reinforcement learning (DRL) for adaptive resource scheduling, providing invaluable insights for our research on scheduling in heterogeneous environments. This paper examines various DRL techniques that enable intelligent, self-learning schedulers capable of making resource allocation decisions dynamically and autonomously. The authors highlight the potential of DRL to optimize resource utilization and adapt to complex, multi-dimensional resource environments, an approach well-suited for handling the diverse CPU, GPU, and memory capabilities across nodes in our project. Drawing from the principles discussed by Zhou et al., our project leverages DRL-inspired methods to improve scheduling flexibility and precision, ensuring that each task is allocated to the optimal node based on its computational requirements, which ultimately enhances both efficiency and scalability in distributed machine learning environments.

Existing research emphasizes the importance of adaptive resource scheduling in heterogeneous computing environments, where diverse hardware resources must be efficiently allocated. Reinforcement learning and deep reinforcement learning have emerged as powerful tools for dynamic and autonomous task allocation, continuously optimizing scheduling decisions based on workload variations. Containerization technologies like Docker further enhance resource sharing and scheduling flexibility by isolating workloads and managing dependencies effectively. Additionally, real-time performance improvements, such as latency reduction and optimized resource utilization, are crucial for distributed machine learning setups. Decentralized scheduling frameworks play a key role in scaling task allocation across dynamic environments, yet challenges such as handling resource heterogeneity, maintaining consistency, and managing execution dependencies remain. Addressing these challenges requires intelligent decision-making strategies combined with robust scheduling methodologies to enhance computational efficiency and adaptability in complex distributed systems.

III. Methodology

A. System Architecture



The architecture is divided into three main phases:

- Phase 1 : The script is decomposed into smaller executable snippets.Each snippet is labeled with its execution order and resource requirements by the Labeller and Orderer module.
- Phase 2 : The labeled snippets are forwarded to the master node.The master node prepares a schedule for execution based on dependencies and resource constraints.
- Phase 3 : The master node establishes a shared environment across distributed nodes.Nodes execute assigned snippets, synchronizing their states with the shared environment to ensure consistency.The shared environment mimics a Jupyter kernel, allowing nodes to execute code cells with access to shared variables and state.

B. Description of the Datasets and tools used to collect them

To support system evaluation and simulate realistic workload scenarios in distributed machine learning environments, a custom-designed profiling dataset was developed. This dataset plays a critical role in assessing the performance of the proposed resource-aware scheduling algorithms by enabling pre-execution classification and optimal resource allocation across heterogeneous compute clusters.

1) Dataset Overview: The dataset comprises 562 labeled entries, each representing a distinct programming keyword or machine learning (ML)-related operation. These keywords are annotated based on their computational dependencies, specifically indicating whether execution on CPU, GPU, or TPU hardware is required or beneficial. This mapping allows the scheduler to predict and allocate the most suitable hardware resources for a given workload.

- **Total Records:** 562
- **Attributes:**
  - **Keyword:** Represents a functional unit or code-level task (e.g., `CNN`, `preprocess`, `import`)
  - **CPU:** Binary indicator (1 or 0) denoting whether CPU computation is involved
  - **GPU:** Binary indicator denoting whether GPU acceleration is advantageous
  - **TPU:** Binary indicator specifying TPU compatibility
- **Purpose:** Enables static analysis of code blocks for pre-runtime classification and resource mapping. This is instrumental for the master node to dynamically assign workloads to appropriate resources within a heterogeneous cluster.

TABLE I
EXAMPLE DATASET ENTRIES

Keyword	CPU	GPU	TPU
CNN	0	1	1
RNN	1	0	1
preprocess	1	1	0

2) Data Collection Tools and Infrastructure:

- **Programming Language:** Python
- **Libraries Used:**
  - **NumPy** for efficient numerical data handling
  - **Pandas** for structured data processing and transformation
  - **TensorFlow** and **PyTorch** for simulating various ML workloads and profiling resource requirements
- **Infrastructure:** A custom-built cloud environment equipped with distributed CPU and GPU nodes was utilized. This infrastructure allowed for controlled emulation of heterogeneous execution scenarios.

3) Limitations: While the dataset provides a robust static profiling mechanism, the system's performance can be influenced by real-time resource availability and variability in hardware characteristics across nodes. Consequently, fluctuations in execution time may occur due to dynamic load balancing and the heterogeneous nature of the compute environment.

C. Description of the Algorithm

The proposed algorithm incorporates both static and dynamic phases, working in tandem to ensure efficient task allocation and execution. The key aspects of the methodology are detailed below:

1) Static Phase - Code Separator: The static phase begins with preprocessing the client code, which refers to the set of instructions or scripts submitted for execution. This phase segments the code into logical cells, similar to the structure of Jupyter notebooks. Each cell represents an independently executable unit while maintaining its role within the overall sequence of the code.

To determine the resource requirements for these cells, keyphrase-detecting Natural Language Processing (NLP) techniques are employed. These techniques analyze the content of the cells to identify sections of the code that might benefit from or require external resources such as GPUs or TPUs. Additionally, the static phase determines whether specific blocks can be executed in parallel, enabling potential performance gains through concurrent execution. Each cell is then labeled with its resource requirements and execution order dependencies, preparing it for the dynamic phase.

2) Dynamic Phase - Master-Slave Architecture: The dynamic phase is executed using a Master-Slave architecture, where the Master Node serves as the central authority for task management and resource allocation. The labeled code blocks from the static phase are passed to the Master Node, which performs the following functions:

- Node Creation and Management: The Master Node creates Slave Nodes equipped with the necessary external resources (e.g., GPUs, TPUs) on-demand. These Slave Nodes are dynamically instantiated and destroyed based on workload requirements.
- Execution Order Control: The Master Node ensures that the code blocks are executed in the correct order, adhering to the dependencies identified during the static phase.
- Result Aggregation: For code blocks that can be executed in parallel, the Master Node aggregates results from the Slave Nodes and ensures that outputs are synchronized for further processing.

This architecture allows for efficient utilization of external resources while maintaining control over the execution process, reducing idle times and optimizing resource usage.

3) Environment: The environment for execution is designed to ensure seamless integration and compatibility among all nodes. Similar to Jupyter notebooks, all code cells share the same execution context, ensuring consistency across nodes. By maintaining a unified execution environment, issues such as mismatched dependencies, missing variables, and lack of sequentiality between nodes are effectively mitigated. This ensures that the distributed execution retains the logical flow of the original code while enabling scalability and efficiency.

IV. Observation

This section presents the observed performance of our resource-aware scheduling system under two different environments: homogeneous and heterogeneous. The key metric evaluated is execution time, with comparisons between normal and segmented execution strategies.

A. Homogeneous Environment

In the homogeneous setup, all computations were executed on a single machine equipped with an NVIDIA T4 GPU. The system was evaluated using a Convolutional Neural Network (CNN) trained on the CIFAR-10 dataset, as well as a reinforcement learning (RL) based model. This environment represents a uniform computational setup without variability in available hardware resources.

Table II shows the execution time comparison for both types of workloads.

TABLE II
Execution Time in Homogeneous Environment (T4 GPU)

Task	Normal Execution (s)	Segmented Execution (s)
CNN	57.77	52.53
RL	209.36	194.27

The use of segmented execution led to a time reduction of approximately 5.24 seconds for the Convolutional Neural Network (CNN) training task, indicating improved efficiency even in uniform environments with consistent hardware capabilities. For the reinforcement learning workload, the execution time decreased from 209.36 seconds in normal execution to 194.27 seconds with segmented execution. This demonstrates that even for iterative and state-dependent tasks such as RL, the resource-aware scheduling algorithm can yield performance improvements when effectively parallelized and optimized for the available hardware.

B. Heterogeneous Environment

In the heterogeneous setup, the execution environment consisted of multiple nodes with varied computational resources, including an NVIDIA L4 GPU. Multiple Python scripts with differing computational characteristics were executed, both normally and with segmented execution, to observe improvements enabled by our scheduling algorithm.

The results are summarized in Table III.

TABLE III
Execution Time in Heterogeneous Environment (L4 GPU)

Target Script Name	Normal Execution(s)	Segmented Execution(s)
`simple`	0.0460	0.0261
`branching`	0.0752	0.0137
`looping`	0.0316	0.0033
`parallelizable`	0.0614	0.0373

These results demonstrate significant performance gains from segmented execution, particularly in scripts with branching logic or iterative operations. The improvement in such cases is attributed to better task parallelization and dynamic resource assignment facilitated by the scheduling algorithm. The heterogeneous nature of the environment, including access to a high-performance L4 GPU, allowed the system to efficiently distribute workloads based on node capabilities.

V. Future Scope

The proposed resource-aware scheduling system has demonstrated efficiency in reducing execution times for distributed machine learning tasks in heterogeneous environments. However, there remains scope to enhance its capabilities by exploring real-time scheduling algorithms that dynamically adapt to varying workloads and resource availability. Incorporating emerging hardware like TPUs and FPGA into the scheduling framework can also improve performance for specific machine learning tasks, enabling broader applicability in diverse computing environments.

Additionally, future work could focus on integrating green computing practices to minimize energy consumption during resource allocation. Expanding compatibility to support multiple machine learning frameworks such as TensorFlow and PyTorch would increase versatility, while incorporating AI-driven techniques like reinforcement learning can optimize scheduling decisions further. Enhancing the system for fault tolerance and scalability across larger clusters is another promising direction to ensure robustness and applicability in industrial-scale deployments.

Acknowledgment

We are sincerely thankful to Dwarkadas J Sanghvi College of Engineering for providing us with the opportunity to conduct research on the project titled ”Resource-Aware Scheduling Algorithms For Distributed Machine Learning in Heterogeneous Environments” . We would like to express my deepest gratitude to my project guide, Dr. Vinaya Sawant, for her invaluable guidance, support, and encouragement throughout the course of this research. Her expertise and insights have been instrumental in shaping the direction and scope of this project. We are sincerely thankful for her constructive feedback and for providing us with the resources and knowledge necessary to accomplish this work. This research would not have been possible without her mentorship and unwavering support.

REFERENCES

[1] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM), 2019, pp. 270–288.

[2] D. Das, A. Tiwari, and D. Agrawal, “Dynamic resource allocation in cloud computing using a distributed scheduler,” in Journal of Cloud Computing, vol. 9, no. 3, pp. 1–10, 2020.

[3] A. V. Vasilakos and M. Liu, “Scheduling and resource allocation for real-time streaming services in cloud computing,” in IEEE Transactions on Cloud Computing, vol. 4, no. 1, pp. 1–10, 2016.

[4] G. Zhou, W. Tian, and R. Buyya, “Deep Reinforcement Learning-based Methods for Resource Scheduling in Cloud Computing: A Review and Future Directions,” in ACM Computing Surveys, vol. 54, no. 3, pp. 1–36, 2021.

[5] A. V. Vasilakos and M. Liu, “Scheduling and resource allocation for real-time streaming services in cloud computing,” in Journal of Cloud Computing, vol. 3, no. 2, pp. 25–38, 2016.

[6] ”A Resource-Aware Scheduling Algorithm with Reduced Task Duplication on Heterogeneous Computing Systems”H. Topcuoglu, S. Hariri, and M. Wu , published in Journal of Supercomputing 1 June 2014.

[7] ”Heterogeneous Computing Systems”A. K. Singh, A. K. Bhunia, and B. Veeravalli.

[8] Z. Zeng, X. Sun, and Y. Chen, ”A container-based scheduling approach for heterogeneous cloud environments,” in IEEE Transactions on Cloud Computing, vol. 8, no.2, pp. 351–364, 2020.

[9] Cifar Dataset :- https://www.kaggle.com/c/cifar-10